

Flora Yamaguti

Ana Beatriz Costa Viana
Grazielle Sorrentino Santos Souza
Gustavo Anselmo Santos Silva
Karina Oliveira Bicalho de Almeida
Nicole Marie Agnelo Marques,

Instituto de Informática e Ciências Exatas – Pontifícia Universidade Católica de
Minas Gerais (PUC MINAS)

Belo Horizonte, MG – Brasil – 2026

abcviana@sga.pucminas.br

1441988@sga.pucminas.br

gustavosilvasocial@gmail.com

1440058@sga.pucminas.br

1478059@sga.pucminas.br

Resumo. *A Flora Yamaguti é um micro-empreendimento familiar que opera no ramo da comercialização de flores, produtos naturais, ornamentais, entre outros. O objetivo do trabalho é auxiliar o controle dos pedidos, criar orçamentos e organizar o catálogo de produtos.*

1. Introdução

Atualmente, a digitalização de pequenos negócios tem se tornado cada vez mais importante para melhorar a eficiência e a competitividade no mercado. Ferramentas tecnológicas, como aplicativos mobile, ajudam a otimizar processos, reduzir erros manuais e organizar melhor as informações, principalmente em micro e pequenas empresas que ainda utilizam métodos mais informais de gestão.

Nesse cenário, o setor de comércio de flores e produtos naturais também vem sentindo a necessidade de se modernizar. Atividades como o controle de pedidos, a criação de orçamentos e a organização do catálogo de produtos são essenciais para o funcionamento do negócio. Porém, quando feitas manualmente, podem gerar retrabalho, inconsistências e até perda de oportunidades.

Diante disso, este trabalho propõe o desenvolvimento de uma aplicação para a Flora Yamaguti, um micro empreendimento familiar que atua na comercialização de flores, produtos naturais e ornamentais. A ideia surge da necessidade de melhorar o controle das atividades do dia a dia, oferecendo uma solução mais prática e centralizada para organizar essas informações.

Assim, o projeto busca minimizar problemas causados pela falta de organização dos processos internos, contribuindo para uma gestão mais eficiente, organizada e com maior potencial de crescimento.

1.1. Objetivos geral e específicos

O objetivo geral é criar um aplicativo que auxilie a dona do comércio com suas tarefas diárias, como controle dos pedidos, criar orçamentos e organizar o catálogo de produtos e fornecedores.

O objetivo específico é projetar e criar uma aplicação offline utilizando as tecnologias de React Native e TypeScript.

1.2. Justificativas

O trabalho se justifica ao contribuir com um pequeno empreendimento na sua organização e ao mesmo tempo permitindo que os integrantes do grupo coloquem em prática seus conhecimentos desenvolvidos durante a disciplina.

1.3. Público-alvo

O público alvo é a proprietária da Flora, Terezinha Kaori Yamaguti, e possíveis futuros funcionários.

1.4. Especificação do Projeto

Na especificação do projeto estão definidas as personas principais para o app, os requisitos definidos após entendimento das dores principais da cliente e o planejamento do cronograma de produção e de recursos.

1.4.1 Personas:

- **Terezinha Kaori Yamaguti (Persona principal)**

Idade: 52 anos

Profissão: Proprietária da Flora Yamaguti

Perfil: Empreendedora, organizada, mas pouco familiarizada com tecnologia

Rotina: Cuida das vendas, atende clientes, organiza pedidos e faz orçamentos manualmente (papel ou WhatsApp).

Dores: Dificuldade em controlar todos os pedidos, perda de informações importantes e retrabalho ao refazer orçamentos.

Objetivos: Organizar melhor os pedidos, criar orçamentos de forma mais rápida e ter tudo em um só. lugar

- **Funcionária auxiliar**

Idade: 20 anos

Profissão: Atendente da loja

Perfil: Mais familiarizado com tecnologia

Rotina: Ajuda no atendimento, anota pedidos e consulta produtos

Dores: Falta de padronização nas informações e dificuldade em acessar dados atualizados.

Objetivos: Acessar pedidos rapidamente, consultar catálogo com facilidade e evitar erros no atendimento.

1.4.2 Histórias de usuário:

- Como proprietária, eu quero registrar pedidos no sistema para não perder informações dos clientes.
- Como funcionário, eu quero acessar os pedidos rapidamente para atender o cliente com mais agilidade.
- Como proprietária, eu quero cadastrar produtos no sistema para manter o catálogo organizado.
- Como funcionário, eu quero consultar o catálogo facilmente para informar os clientes.
- Como usuária, eu quero centralizar todas as informações em um só lugar para evitar confusão.
- Como usuária, eu quero uma interface simples para conseguir usar o sistema sem dificuldade.
- Como proprietária, eu quero exportar um backup dos dados para não correr risco de perder informações.

1.4.3 Requisitos funcionais e não funcionais

ID	Requisito	Prioridade	Implementado
RF1	Manter um fornecedor (visualizar, criar, editar, excluir)	Alta	Sim
RF2	Manter um produto (visualizar, criar, editar, excluir)	Alta	Sim
RF3	Listar produtos por fornecedor e preço	Alta	Sim
RF4	Listar fornecedores	Alta	Sim
RF5	Manter um pedido (visualizar, criar, editar, excluir)	Alta	Sim
RF6	Gerar valor estimado do pedido com base nos produtos	Alta	Sim
RF7	Alterar o status do pedido entre “Em preparo”, “Pendente” ou “Entregue”	Média	Sim
RF8	Visualizar o histórico de pedidos	Média	Sim
RF9	Exportar os dados de qualquer etapa em um arquivo pdf.	Média	Não
RF10	Exportar os dados de fornecedores, produtos e pedidos em um arquivo .json	Média	Sim
RF11	Registrar alterações da sessão para backup manual	Média	Sim

ID	Requisito Não Funcional	Prioridade	Implementado
RNF1	A aplicação deve operar offline.	Alta	Parcial*
RNF2	Interface responsiva para celular e tablet	Média	Sim
RNF3	Formulários com padrão de cores, design e botões uniforme	Baixa	Sim
RNF4	Controle granular de permissões de usuários	Alta	Não
RNF5	Feedbacks com mensagens claras para operações de sucesso ou falha	Média	Sim

*RNF1: O Firestore SDK armazena dados em cache local via IndexedDB. O app opera offline a partir da segunda sessão; a primeira sessão requer conectividade para popular o cache.

1.4.4 Planejamento de cronograma

O desenvolvimento do projeto será dividido em etapas, organizadas de forma sequencial ao longo do período da disciplina:

- **Levantamento de requisitos:** identificação das necessidades do usuário e definição das funcionalidades principais
- **Planejamento e modelagem:** criação de personas, histórias de usuário e estrutura do sistema
- **Prototipação:** desenvolvimento de telas e definição da interface do aplicativo
- **Desenvolvimento:** implementação da aplicação utilizando React Native e TypeScript
- **Testes:** validação das funcionalidades e correção de possíveis erros
- **Entrega final:** ajustes finais e documentação do projeto

1.4.5 Custos e pessoal do projeto

O projeto foi desenvolvido pelos integrantes do grupo, sem custos diretos com contratação de equipe externa. Todas as ferramentas utilizadas são gratuitas: Vite, React, Electron, Capacitor e Firebase (plano Spark — gratuito). A hospedagem do banco de dados é fornecida pelo Firebase sem custo para o volume de dados desta aplicação. Os custos indiretos limitam-se ao tempo dedicado pelos integrantes e aos recursos já disponíveis (computadores pessoais, acesso à internet).

2. Participantes do processo de negócio

Os stakeholders do sistema da Flora Yamaguti são identificados conforme seus papéis no processo de negócio e sua relação direta ou indireta com o aplicativo:

1. Proprietária · Usuária Principal

- Terezinha Kaori Yamaguti

Interação com o sistema

- Usuária direta e administradora de facto. Cadastra fornecedores e produtos, registra pedidos, altera status, consulta histórico e realiza backups.

Interesse

- Ter controle centralizado dos pedidos e orçamentos; reduzir retrabalho; agilizar o atendimento.

Nível de acesso

- Total — leitura e escrita em todas as entidades do sistema.

2. Atendente · Usuária Secundária

- Funcionária Auxiliar

Interação com o sistema

- Consulta pedidos em aberto e o catálogo de produtos durante o atendimento ao cliente. Pode registrar novos pedidos quando a proprietária não está presente.

Interesse

- Agilidade no atendimento; acesso rápido a preços estimados e disponibilidade de produtos.

Nível de acesso

- Leitura geral; escrita restrita a pedidos. Nota: o controle granular de permissões (RNF4) não foi implementado nesta versão — ambas as usuárias acessam o sistema com as mesmas permissões.

3. Parceiros Comerciais · Externos ao sistema

- Fornecedores

Interação com o sistema

- Não utilizam o aplicativo diretamente. Seus dados (nome, contato, produtos fornecidos, preços e unidades) são cadastrados pela proprietária para referência interna.

Interesse

- Que as informações de preço e disponibilidade sejam registradas corretamente.

4. Equipe de Desenvolvimento

- Grupo 4 - PUC Minas 2026

Interação com o sistema

- Responsáveis por projetar, desenvolver e manter o aplicativo ao longo do período da disciplina.

Interesse

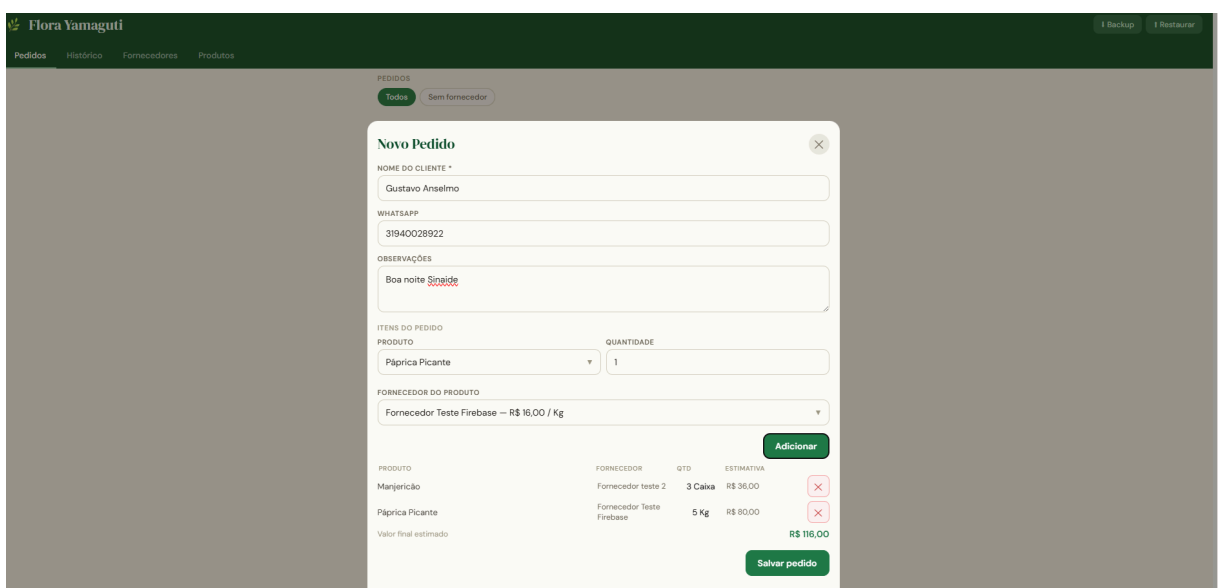
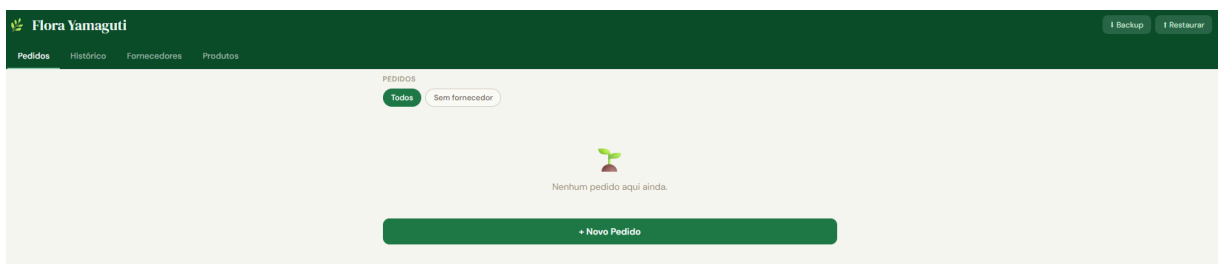
- Entregar uma solução funcional que atenda às necessidades da cliente e demonstre os conhecimentos adquiridos na disciplina.

3. Projeto da Solução

A interface é organizada em quatro abas de navegação fixas no topo da tela. Abaixo, a descrição de cada tela e suas funcionalidades principais.

3.1. Protótipos de telas

- Pedidos: Tela principal do app. Lista todos os pedidos ativos com nome do cliente, data, status e valor estimado. Permite criar novo pedido (cliente, observações, itens com produto, fornecedor e quantidade), editar, excluir e alterar o status (Pendente → Em preparo → Entregue). Pedidos marcados como "Entregue" são movidos automaticamente para o Histórico.



PEDIDOS

Todos

Sem fornecedor

Fornecedor teste 2

Fornecedor Teste Firebase

Gustavo Anselmo

21/06/2026 · 31940028922

Pendente

Manjeriç o

Fornecedor teste 2 · Estimativa: R\$ 36,00

3 Caixa

P aprica Picante

Fornecedor Teste Firebase · Estimativa: R\$ 80,00

5 Kg

Valor final: R\$ 116,00

Boa noite Sinaide

Fornecedor teste 2 Fornecedor Teste Firebase

Pendente

Editar

Excluir

+ Novo Pedido

PEDIDOS

Todos

Sem fornecedor

Fornecedor teste 2

Fornecedor Teste Firebase

Gustavo Anselmo

21/06/2026 · 31940028922

Em preparo

Manjeriç o

Fornecedor teste 2 · Estimativa: R\$ 36,00

3 Caixa

P aprica Picante

Fornecedor Teste Firebase · Estimativa: R\$ 80,00

5 Kg

Valor final: R\$ 116,00

Boa noite Sinaide

Fornecedor teste 2 Fornecedor Teste Firebase

Em preparo

Editar

Excluir

+ Novo Pedido

- Histórico: Exibe pedidos finalizados agrupados por cliente. Para cada grupo mostra o total de pedidos e itens. Cada pedido exibe os itens, fornecedores e o valor total estimado na época da entrega.

Flora Yamaguti

Pedidos Histórico Fornecedores Produtos

HISTÓRICO POR CLIENTE

GA **Gustavo Anselmo**
2 pedidos · 12 itens no total

21/06/2026 **Entregue**

Manjeriçã
Fornecedor teste 2 **3 Caixa**

Páprica Picante
Fornecedor Teste Firebase **5 Kg**

Valor do pedido: R\$ 116,00

Boa noite Sinaide

21/06/2026 **Entregue**

Manjeriçã
Fornecedor teste 2 **1 Caixa**

Páprica Picante
Fornecedor Teste Firebase **3 Kg**

Valor do pedido: R\$ 60,00

Teste Firebase

GA **Gustavo Anselmo**
2 pedidos · 12 itens no total

21/06/2026 **Entregue**

Manjeriçã
Fornecedor teste 2 **3 Caixa**

Páprica Picante
Fornecedor Teste Firebase **5 Kg**

Valor do pedido: R\$ 116,00

Boa noite Sinaide

21/06/2026 **Entregue**

Manjeriçã
Fornecedor teste 2 **1 Caixa**

Páprica Picante
Fornecedor Teste Firebase **3 Kg**

Valor do pedido: R\$ 60,00

Teste Firebase

- Fornecedores: Cadastro de fornecedores com nome, contato e lista de produtos oferecidos. Cada produto tem nome, preço estimado (opcional) e tipo de unidade (Kg, Caixa, Vaso, Unidade ou Outro). Validações em linha impedem salvar sem nome ou sem pelo menos um produto.

Cadastrar fornecedor

QUAL FORNECEDOR

Vox Machina

PRODUTOS + Produto

PRODUTO

Rosa

QTD TIPO ESTIMATIVA

Vaso ▼

27

Cancelar inserção

PRODUTO

Adubo

QTD TIPO ESTIMATIVA

Kg ▼

12

Cancelar inserção

NÚMERO PARA CONTATO DO FORNECEDOR

31123456789

Salvar fornecedor

FORNECEDORES CADASTRADOS (3)

Fornecedor Teste Firebase

31998050847

Páprica Picante (Kg) · R\$ 16,00

Editar

Excluir

Fornecedor teste 2

31998050847

Manjerição (Caixa) · R\$ 12,00

Editar

Excluir

Vox Machina

31123456789

Rosa (Vaso) · R\$ 27,00

Adubo (Kg) · R\$ 12,00

Editar

Excluir

Flora Yamaguti

Backup Restaurar

Pedidos Histórico Fornecedores Produtos

PRODUTOS

+ Produto

PRODUTO

Ex: Manjerição

QTD TIPO

ESTIMATIVA

Unidade

R\$ (opcional)

Cancelar inserção

NÚMERO PARA CONTATO DO FORNECEDOR

(11) 99999-0000

Salvar fornecedor

FORNECEDORES CADASTRADOS (3)

Fornecedor Teste Firebase

31998050847

Páprica Picante (Kg) · R\$ 16,00

Editar

Excluir

Fornecedor teste 2

31998050847

Manjerição (Caixa) · R\$ 12,00

Editar

Excluir

Vox Machina

31123456789

Rosa (Vaso) · R\$ 27,00

Adubo (Kg) · R\$ 12,00

Editar

Excluir

- **Produtos:** Visão consolidada de todos os produtos cadastrados nos fornecedores, exibida em tabela com produto, fornecedor, contato, unidade e preço estimado. Somente leitura — o cadastro é feito pela tela de Fornecedores.

Produtos por fornecedor (visualização)

Esta tela mostra somente consulta. Cadastre e edite os dados na aba Fornecedores.

PRODUTOS CADASTRADOS (4)

PRODUTO	FORNECEDOR	CONTATO	UNIDADE	PREÇO ESTIMADO
Adubo	Vox Machina	31123456789	Kg	R\$ 12,00
Manjerição	Fornecedor teste 2	31998050847	Caixa	R\$ 12,00
Páprica Picante	Fornecedor Teste Firebase	31998050847	Kg	R\$ 16,00
Rosa	Vox Machina	31123456789	Vaso	R\$ 27,00

Flora Yamaguti Backup Restaurar

Pedidos Histórico Fornecedores **Produtos**

Produtos por fornecedor (visualização)

Esta tela mostra somente consulta. Cadastre e edite os dados na aba Fornecedores.

PRODUTOS CADASTRADOS (4)

PRODUTO	FORNECEDOR	CONTATO	UNIDADE	PREÇO ESTIMADO
Adubo	Vox Machina	31123456789	Kg	R\$ 12,00
Manjerição	Fornecedor teste 2	31998050847	Caixa	R\$ 12,00
Páprica Picante	Fornecedor Teste Firebase	31998050847	Kg	R\$ 16,00
Rosa	Vox Machina	31123456789	Vaso	R\$ 27,00

3.1.1. Processos e Propriedades

Processo 1: Cadastro e Gestão de Fornecedores

Atividade: Cadastrar, editar e excluir fornecedores e seus produtos.

Nome	Nome da propriedade ou campo	Tipo
Fornecedor	id	string (gerado por genId())
Fornecedor	nome	string
Fornecedor	contato	string

FornecedorProduto	nome	string
FornecedorProduto	precoEstimado	number null
FornecedorProduto	unidadeTipo	"Kg" "Caixa" "Vaso" "Unidade" "Outro"
FornecedorProduto	unidadeOutro	string (quando unidadeTipo = "Outro")

Processo 2: Cadastro e Acompanhamento de Pedidos

Atividade: criar pedidos, atualizar status e consultar histórico.

Nome	Propriedade	Tipo
Pedido	id	string
Pedido	cliente	string (nome do cliente, embutido no pedido)
Pedido	contato	string
Pedido	obs	string
Pedido	data	string (dd/mm/aaaa)
Pedido	status	"pendente" "em_preparo" "entregue"
Pedido	dataEntrega	string
PedidoItem	produto	string (nome do produto)
PedidoItem	qtd	number

PedidoItem	fornId	string (ref ao id do Fornecedor)
PedidoItem	unidade	string

Processo 3: Consulta de Produtos e Relatorios

Atividade: consultar produtos por fornecedor e exportar/importar backup.

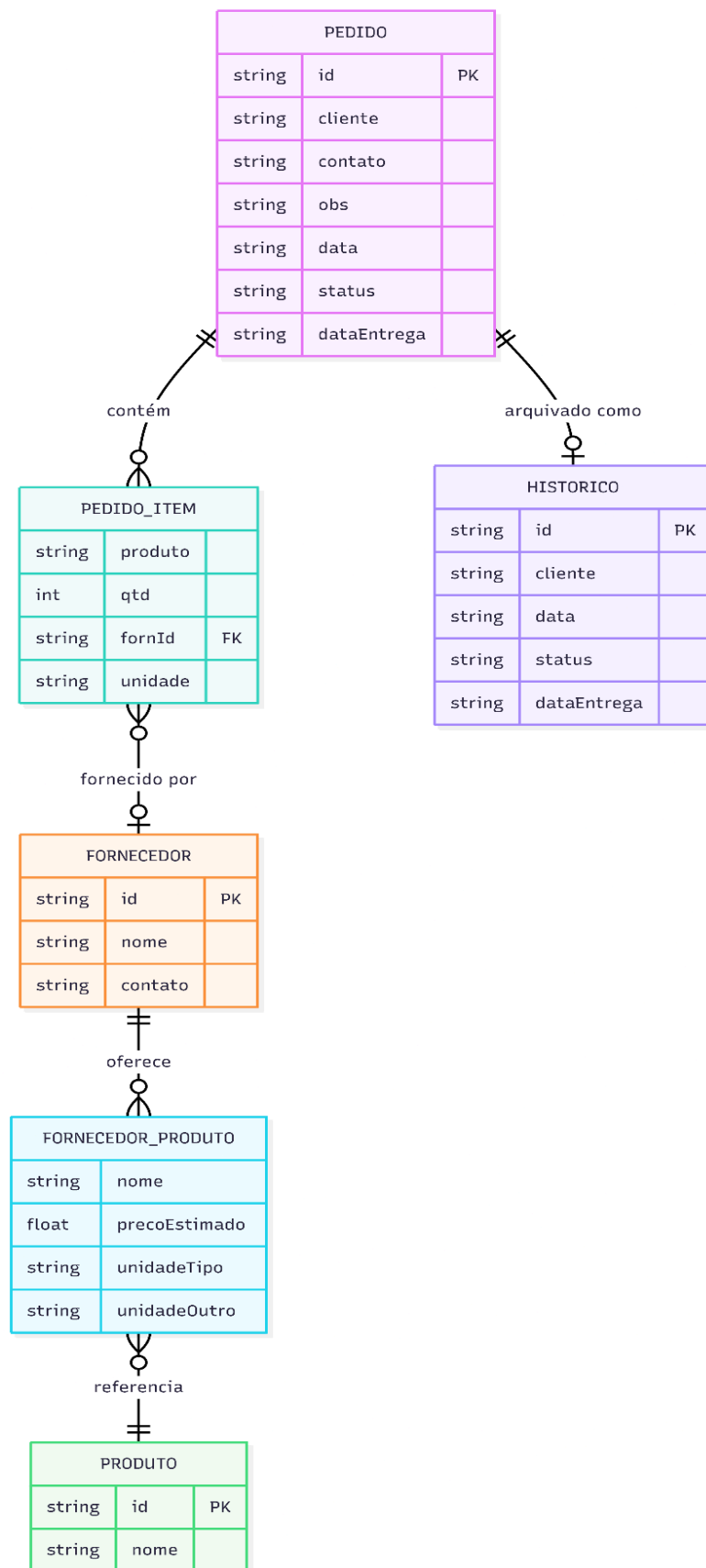
Nome	Nome da propriedade ou campo	Tipo
Produto	id	string (sincronizado a partir de Fornecedor)
Produto	nome	string
BackupJSON	pedidos	Pedido[]
BackupJSON	fornecedores	Fornecedor[]
BackupJSON	produtos	Produto[]
BackupJSON	historico	Pedido[]
BackupJSON	exportadoEm	string (data/hora)

3.2. Diagrama Entidade e Relacionamento (DER)

O modelo de dados é document-oriented (Firebase Firestore), com entidades embutidas dentro de seus agregados para minimizar leituras ao banco. O diagrama abaixo representa a estrutura lógica das entidades e seus relacionamentos.



Nota: não há entidade Cliente separada — o cliente é representado apenas pelo nome e contato embutidos em cada Pedido. O controle de acesso por Usuário/Papel (Processo 4 do planejamento original) não foi implementado nesta versão.



3.3. Tecnologias

Camada	Tecnologia Proposta	Versão	Motivo
Front-end web/app	React + Vite + JavaScript	19 / 8	Estrutura moderna de SPA com hot reload rápido; JavaScript (sem TypeScript) para reduzir configuração
Banco de dados em nuvem	Firebase Firestore	11	NoSQL document-oriented com sincronização em tempo real e cache offline automático via IndexedDB (persistentLocalCache)
Persistência complementar	JSON (backup/restauracao)	-	Exportação e importação manual de todos os dados para segurança adicional
Desktop runtime	Electron	31	Empacotamento e execução como app desktop Windows (.exe via electron-builder)
Mobile runtime	Capacitor	8	Build Android nativo a partir do código React existente
Build e dependências	Node.js + npm	-	Execução e gerenciamento de pacotes
IDE	Visual Studio Code	-	Desenvolvimento e manutenção

4. Uso Software

O aplicativo Flora Yamaguti é uma Single Page Application (SPA) React distribuída em duas plataformas: desktop (Electron / Windows) e mobile (Capacitor / Android). A interface é idêntica nas duas plataformas — o Electron carrega o build do Vite em uma janela nativa e o Capacitor empacota o mesmo build como APK Android.

4.1. Front-end

O front-end é inteiramente construído em React com JavaScript. Não há roteador externo — a navegação é feita por controle de estado de aba (activeTab) no componente raiz. Toda a estilização é feita com objetos de estilo inline usando dois objetos de tokens centralizados em src/tokens.js: T (paleta de cores) e S (objetos de estilo reutilizáveis), sem arquivos CSS externos nem Tailwind.

Estrutura de arquivos:

ENTRADA

index.html → src/main.jsx → <App />

O Electron aponta para renderer/index.html (output do build do Vite).

CONTEXTO

src/context/AppContext.jsx – expõe AppProvider (envolve a aplicação) e o hook useApp(), que retorna toda a lógica e dados de negócio via React Context. Elimina prop drilling em todos os componentes de tela.

LÓGICA

src/hooks/useAppData.js – hook central com todo o estado e operações CRUD: salvarPedido, excluirPedido, mudarStatus, salvarFornecedor, excluirFornecedor, exportarDados, importarDados, além de funções de consulta como getPrecoEstimado, calcTotal e listarProdutosDisponiveis. Expõe também o flag loading (verdadeiro até todos os dados serem carregados do Firestore).

TELAS

PedidosScreen, HistoricoScreen, FornecedoresScreen e ProdutosScreen.

Cada tela usa useApp() para acessar dados e funções, sem receber props. Contém estado local apenas para controle de formulários (campos, erros, modais de confirmação).

COMPONENTES

Badge, Chip, Btn, FormGroup, StyledInput, StyledSelect, StyledTextarea, Modal, ConfirmModal, FormError e SectionLabel.

Componentes visuais atômicos centralizados em src/components/index.js (barrel export). Todos recebem props explícitas; nenhum acessa o contexto diretamente.

UTILITÁRIOS

src/constants.js – chaves de storage, mapa de status, opções de unidade.

src/utlis.js – formatCurrency, parsePrice, getInitials, genId.

src/normalize.js – normalizeFornecedores, normalizeProdutos, syncProdutos (migração de dados legados e sincronização de referências fornecedor→produto).

Padrão de navegação:

A navegação é gerenciada por um array TABS no componente AppContent e um estado activeTab. Cada aba renderiza condicionalmente a tela correspondente com operadores &&.

Não há React Router, pois o app não tem URLs navegáveis — é uma aplicação desktop/mobile de tela única.

Tratamento de erros e feedback ao usuário:

Os alerts e confirms nativos do navegador foram substituídos por componentes próprios: FormError exibe mensagens de validação inline abaixo de cada campo; ConfirmModal substitui o window.confirm() com um modal customizado; um componente de toast (estado no AppContent) exibe notificações temporárias de 3,5 segundos para operações de backup e restauração.

```
// Exemplo: hook useApp() consumido diretamente pela tela
export function PedidosScreen() {
  const { pedidos, salvarPedido, excluirPedido, calcTotal } = useApp();
  // sem props - contexto elimina o prop drilling
  ...
}
```

4.1. Back-end

O back-end da aplicação é o Firebase Firestore, um banco de dados NoSQL orientado a documentos com suporte a sincronização em tempo real e cache offline automático. Não há servidor próprio, API REST, nem lógica de negócio fora do cliente.

Estrutura no Firestore

Todos os dados residem em uma única coleção flora-data, com quatro documentos — um por entidade principal — cada um contendo um campo value com o array de registros:

Coleção	Documento	Campo	Conteúdo
flora-data	plt_pedidos	value	Array de Pedido[]
flora-data	plt_fornecedores	value	Array de Fornecedor[]
flora-data	plt_produtos	value	Array de Produto[]
flora-data	plt_historico	value	Array de Pedido[] (entregues)

Hook useLocalStorage (camada de persistência)

O hook src/hooks/useLocalStorage.js abstrai completamente o acesso ao Firestore. Ele expõe a mesma interface [state, set, ready] que o hook anterior baseado em localStorage,

permitindo que o `useAppData.js` e as telas não precisem saber qual tecnologia de persistência está sendo usada.

```
// Inicialização do Firestore com cache offline
export const db = initializeFirestore(app, {
  localCache: persistentLocalCache(), // IndexedDB
});

// Hook de persistência – interface idêntica ao localStorage anterior
export function useLocalStorage(key, initial) {
  const [state, setState] = useState(initial);
  const [ready, setReady] = useState(false);

  useEffect(() => {
    const ref = doc(db, "flora-data", key);
    const unsub = onSnapshot(ref, (snap) => {
      if (snap.exists()) setState(snap.data().value);
      setReady(true);
    });
    return unsub; // desinscreve ao desmontar
  }, [key]);

  const set = useCallback((val) => {
    setState((prev) => {
      const next = typeof val === "function" ? val(prev) : val;
      setDoc(doc(db, "flora-data", key), { value: next });
      return next; // atualização otimista
    });
  }, [key]);

  return [state, set, ready];
}
```

Fluxo de dados

- Leitura: `onSnapshot` assina cada documento no Firestore. Quando o app abre, o SDK serve dados do cache IndexedDB imediatamente (sem esperar a rede), e atualiza em segundo plano quando a sincronização completa.
- Escrita: `setDoc` persiste o novo estado no Firestore. O estado React é atualizado de forma otimista (antes da confirmação do servidor), garantindo resposta imediata na UI.
- Offline: escritas feitas sem conexão ficam pendentes na fila local do SDK e são sincronizadas automaticamente quando a conectividade é restabelecida.
- Migração de dados legados: na primeira execução após a migração do `localStorage` para o Firestore, o hook verifica se existe dado no `localStorage` antigo e o migra automaticamente para o Firestore, removendo a entrada legada.

Segurança

O banco de dados usa regras do Firestore em modo de teste (`allow read, write: if request.time < timestamp.date(2026, 7, 21)`), válidas por 30 dias a partir da criação. Para produção, as regras devem ser atualizadas para exigir autenticação Firebase (Authentication), restringindo o acesso apenas às usuárias autorizadas.

5. Conclusão

O desenvolvimento do aplicativo Flora Yamaguti resultou em uma ferramenta funcional que atende às principais necessidades da proprietária: centralizar o controle de pedidos, organizar o catálogo de fornecedores e produtos e agilizar a criação de orçamentos estimados. A migração do processo manual (papel e WhatsApp) para um sistema digital representa um ganho concreto em organização e rastreabilidade das operações diárias do negócio.

Do ponto de vista técnico, o projeto permitiu ao grupo praticar o ciclo completo de desenvolvimento de um aplicativo multiplataforma: da elicitação de requisitos ao deploy em desktop (Electron) e mobile (Capacitor), passando pela organização de uma codebase React com Context API, componentes reutilizáveis, validações de formulário e integração com serviço de nuvem.

Principais Resultados

- CRUD completo de pedidos, fornecedores e produtos;
- Cálculo automático do valor estimado de cada pedido;
- Histórico de pedidos entregues organizado por cliente;
- Persistência de dados em nuvem utilizando o Firebase Firestore;
- Funcionamento offline por meio de cache local com IndexedDB;
- Backup e restauração manual dos dados através de arquivos JSON;
- Feedback de erros em linha (inline), sem utilização de alertas nativos do navegador;
- Interface responsiva, adaptada para uso em dispositivos desktop e mobile.

Limitações do Projeto

- Ausência de autenticação, permitindo que qualquer pessoa com acesso à chave do Firebase possa ler e modificar os dados;
- Falta de controle de permissões por usuário, não atendendo ao requisito não funcional RNF4;
- Exportação de relatórios em PDF não implementada, deixando o requisito funcional RF9 pendente;

- Suporte offline limitado, funcionando apenas a partir da segunda sessão, já que a primeira utilização requer conexão com a internet para carregamento inicial dos dados;
- Regras de segurança do Firestore configuradas em modo de teste, com validade temporária de 30 dias e sem proteção adequada para um ambiente de produção.

REFERÊNCIAS

FACEBOOK INC. React — A JavaScript library for building user interfaces. Disponível em: <<https://react.dev>>. Acesso em: jun. 2026.

GOOGLE LLC. Firebase Firestore Documentation. Disponível em: <<https://firebase.google.com/docs/firestore>>. Acesso em: jun. 2026.

ELECTRON. Electron — Build cross-platform desktop apps with JavaScript, HTML, and CSS. Disponível em: <<https://www.electronjs.org>>. Acesso em: jun. 2026.

IONIC. Capacitor — Cross-platform Native Runtime for Web Apps. Disponível em: <<https://capacitorjs.com>>. Acesso em: jun. 2026.

VITE. Vite — Next Generation Frontend Tooling. Disponível em: <<https://vite.dev>>. Acesso em: jun. 2026.

Github do projeto — *Desenvolvimento-Mobile*. GitHub, 2026. Disponível em: <https://github.com/AnselmoGustavo/Desenvolvimento-Mobile>. Acesso em: jun. 2026.